

电子电路设计报告

基于 FPGA 的虚拟机器人

实时感知、决策与闭环运动控制系统

利用 EG4S20 FPGA、ES8388 实现数字孪生机器人语音交互控制

学 院 机械工程与机器人学院

专 业 智能制造工程

学 号 2452985

姓 名 钟子悠

指导老师 赵亚辉、周伟

日 期 2026 年 9 月

摘 要

本课程设计以安路 EG4S20BG256 FPGA 开发板为核心控制平台，以“感知、决策、执行”为总体设计思想，实现了 FPGA 数字逻辑控制、虚拟执行器建模、语音信号处理、说话人身份认证、命令识别以及 PC 端数字孪生等功能。

系统首先构建了第一阶段运动控制平台。用户通过 FPGA 电路板上的矩阵键盘输入目标角度，系统依次完成目标输入、状态机控制、轨迹规划、PID 调节以及虚拟电机模型计算，同时通过 LED、数码管和 UART 输出系统状态。该部分先通过 ModelSim 仿真，后进行 FPGA 板级验证，实现了目标位置的平滑跟踪、到位保持以及故障演示。

在此基础上，系统进一步扩展语音感知功能。采用 ES8388 音频编解码器进行立体声音频接口设计，建立 48 kHz I2S 音频数据采集链路，并在 FPGA 内部完成语音帧处理、VAD、预加重、分帧加窗、64 点 FFT、Mel 滤波器组、Log 运算和 DCT-II 等处理，形成 13 维 MFCC 特征。

针对“只响应主人指令”的应用需求，将语音处理划分为“Who”和“What”两条并行处理链：Who 链负责说话人身份认证，What 链负责命令识别。为提高特征判别力，系统采用段均值决策替代逐帧决策；同时利用双麦克风的左右声道能量差估计声源方向，使“前进”命令可按声源方向执行左转、右转或直行。

为提高硬件算法实现的可靠性，本设计采用 Python Golden 参考模型、定点 RTL、ModelSim 仿真、FPGA 板级验证的逐级验证方法。MFCC 特征提取链路已经完成 Python 与 RTL 的逐位一致性验证，多个测试场景下输出 diff=0；声纹认证、命令识别以及 Who 及 What 联合决策均完成 RTL 仿真验证。与此同时，PC 端建立了声纹离线评估工具链和 Web 数字孪生系统。

实验结果表明，本系统已经完成从真实语音输入到机器人运动的功能。但是，受 13 维 MFCC 特征表达能力的限制，stop/left/right 等命令词之间的判别精度仍然有限（左、右存在混淆），声纹认证依赖段级均值决策，且模板只在“采集时的声学条件”下有效，跨会话存在声学失配问题。因此本系统定位为面向闭环验证的轻量级原型平台，上述局限正是后续改进的方向。

关键词：FPGA；EG4S20；ES8388；I2S；MFCC；声纹认证；段级决策；命令识别；双麦方向；闭环控制；数字孪生

一、 设计目标

（一） 选题依据

随着 FPGA、数字信号处理和人工智能技术的发展，传统的数字电路课程设计已经逐渐从单一功能模块向多模块协同的综合系统发展。机器人作为典型的机电一体化系统，需要同时具备环境感知、信息处理、决策以及执行能力。因此，将 FPGA 数字逻辑设计与机器人运动控制、语音交互技术相结合，可以形成具有较强综合性的电子电路课程设计课题。

本设计选择“数字孪生机器人的实时感知、决策与运动控制”作为研究对象。一方面，机器人运动控制本身具有明确的闭环结构，可以将目标输入、状态机、轨迹规划、PID 控制器以及执行器模型进行完整串联；另一方面，语音是自然的人机交互方式，如果机器人不仅能够识别命令内容，还能够判断说话人身份，则可以进一步实现“只执行主人指令”的交互方式。

因此，本设计将整个系统抽象为感知、决策、执行三个层次。感知层负责获取目标位置或语音信息，决策层负责状态判断、声纹认证和命令识别，执行层负责轨迹规划、PID 调节以及虚拟电机运动。通过这种模块化设计，可以使不同传感器和算法之间保持相对独立，同时便于后续进行功能扩展。

（二） 设计目的

本课程设计的主要目的包括以下几个方面。

首先，通过实际 FPGA 工程训练掌握数字电路系统从需求分析、模块设计、RTL 编程、仿真验证到 FPGA 上板测试的完整开发流程。通过将系统划分为多个独立模块，可以进一步理解同步时序逻辑、组合逻辑、状态机、参数化设计和模块化接口等 FPGA 设计方法。

其次，通过虚拟电机模型建立一个无需真实机械机构即可验证控制算法的平台。系统利用一阶惯性模型模拟执行器速度响应，通过位置积分得到虚拟机械臂位置，再将位置反馈给 PID 控制器，从而形成完整的闭环控制过程。

再次，将 ES8388 音频接口和语音数字信号处理引入 FPGA，使课程设计不仅包含传统数字控制内容，还能够涉及 I2C、I2S、PCM、FFT、Mel 频谱和 MFCC 等数字信号处理技术。

最后，通过“Who”与“What”的联合决策方式探索更加完整的人机交互逻辑。系统不仅需要回答“说了什么”，还需要判断“是谁说的”，只有在主人身份认证通

过且命令有效的情况下才允许机器人执行动作。

（三） 应用价值

本系统具有一定的工程实践价值。

在智能控制方面，系统可以作为 FPGA 闭环控制实验平台，用于验证目标输入、状态机、轨迹规划、PID 控制和执行器模型之间的关系。

在语音交互方面，声纹认证和命令识别相结合的方式可以应用于需要身份限制的语音控制场景，例如智能家居、实验室设备控制和个人语音终端等。

（四） 课题设计实现的目标及功能

本设计分为两个逐步递进的阶段。

第一阶段为 V1 运动控制系统。其目标是建立完整的 FPGA 闭环控制平台，实现：（1）目标角度输入及合法性检查；（2）IDLE、READY、MOVE、HOLD、FAULT 五状态状态机；（3）目标轨迹限速与平滑规划；（4）定点 PID 位置控制，虚拟电机一阶惯性模型及位置积分；（5） $0^{\circ}\sim 180^{\circ}$ 行程限制；（6）故障检测及人工故障演示；（7）LED、数码管和 UART 状态输出；（8）PC 端数字孪生实时显示。

第二阶段为 V2 语音感知系统。在 V1 的基础上增加：（1）ES8388 音频编解码器接口；（2）48 kHz I2S 音频采集；（3）VAD 活动语音检测；（4）13 维 MFCC 特征提取；（5）基于 MFCC 模板的轻量级声纹认证；（6）stop、left、right、forward 四类命令识别；（7）双麦能量方向感知；（8）Who、What 联合决策；（9）语音命令到机器人目标位置的映射。

其中，V1 系统已经完成 FPGA 板上验证；V2 的音频数字接口、特征提取与决策算法均已完成验证，但受轻量级特征条件限制，板上验证时命令词的判别精度仍有提升空间。

二、 设计方案

（一） 系统总体设计方案

本设计采用模块化、分层式结构。系统总体可以分为 FPGA 核心控制部分、ES8388 音频采集部分以及 PC 数字孪生部分。

FPGA 是系统核心控制器。运动控制链负责完成目标输入、状态控制、轨迹规划、PID 调节和虚拟执行器建模；语音链负责从 ES8388 接收 PCM 音频并进行特征提取、身份认证和命令识别；最终两个方向的信息在联合决策模块中汇合。

其中，运动控制部分既可以直接使用键盘目标输入，也可以由语音命令产生目标位置，从而实现不同输入方式对同一闭环控制系统的复用。

(二) 系统总体结构框图

系统总体结构如图 1 所示。

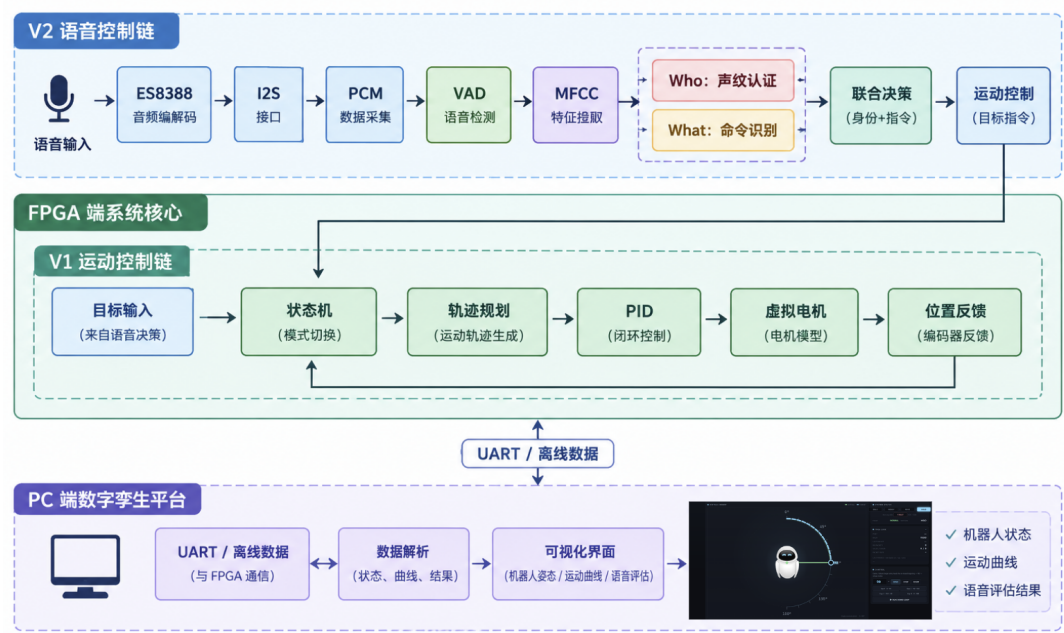


图 1 系统总体结构框图

V1 运动控制链的基本结构为：

目标输入 → 状态机 → 轨迹规划 → PID → 虚拟电机 → 位置反馈

V2 语音控制链则为：

ES8388 → I2S → PCM → VAD → MFCC →
Who：声纹认证；What：命令识别 → 联合决策 → 运动控制

PC 端通过 UART 或离线数据与 FPGA 系统进行通信，用于观察控制状态、运动曲线和语音评估结果。

(三) FPGA 同步数字设计原理

FPGA 内部所有时序逻辑均由板上 50 MHz 晶振经时钟专用网络产生的单一系统时钟 sys_clk 驱动。设计遵循以下原则。

(1) **单一时钟域与时钟使能。**系统不使用逻辑分频产生的新时钟去驱动逻辑，而是用计数器产生周期性的使能脉冲 `tick_1ms`，令其在每 `MS_DIV` 个系统时钟周期拉高一拍。控制节拍为

$$T_{tick} = \frac{MS_DIV}{f_{clk}} = \frac{50000}{50\text{ MHz}} = 1\text{ ms}$$

由于所有寄存器仍工作在 50 MHz 主时钟下，只有使能条件不同，布局布线工具只需分析一个时钟域，避免了分频时钟带来的时钟偏斜、毛刺以及对时序约束的额外要求。

(2) **同步复位。**系统采用低有效同步复位信号 `rst_n`（板上 SW0），复位在时钟沿上生效，时序确定、便于分析。

(3) **时序与组合逻辑严格分离。**时序逻辑统一使用非阻塞赋值，组合逻辑使用 `always @(*)` 或 `assign`，避免在同一块中混用两种赋值造成仿真与综合结果不一致。

(4) **跨时钟域（CDC）处理。**ES8388 以自身产生的位时钟 `aud_bclk` 作为 I2S 主时钟，音频数据在 `aud_bclk` 域进入 FPGA，而后续处理均在 `sys_clk` 域，属于典型的异步跨时钟域。为此，`audio_pcm_bridge` 先在 `aud_bclk` 域内完成串并转换与声道分离，再用一个翻转电平 `pair_tgl` 作握手信号；`sys_clk` 域用两级触发器对该电平同步，再做边沿检测得到 `pair_valid`，在边沿上锁存已经稳定的左右采样。两级同步器把亚稳态传播概率降到可忽略，这是多比特数据跨域的标准做法。

(5) **流水线与复用。**计算量较大的模块（FFT、Mel、DCT）采用多级流水复用结构，在 50 MHz 下以固定周期完成一帧处理。

（四） 定点数值表示与量化原理

FPGA 不适合大量浮点运算，因此本设计内部大量使用定点（fixed-point）数。一个 Q 格式定点数 X 代表的实际值为 $X/2^Q$ 。本设计涉及的定标包括：预加重系数 Q14、Hann 窗与 FFT 旋转因子 Q15、Mel 与 DCT 基系数 Q12。定点设计中有三点关键。

(1) **位宽增长。**两数相乘位宽为二者之和， N 项累加再增加约 $\lceil \log_2 N \rceil$ 位。因此乘法与累加的中间量统一采用 64 位有符号变量，最后再右移定标并截断，避免中间结果溢出。

(2) **符号与溢出。**有符号数右移必须使用算术移位 `>>>`，保证负数向下取整；若误用逻辑右移会把符号位当作数值。当溢出无法避免时应做饱和（钳位到最大/最小值）而不是任其回绕，因为回绕会使符号翻转，产生完全错误的结果。

(3) Q15 的特殊陷阱。均匀量化下 $\cos(0) = 1$ 对应的 Q15 码为 $2^{15} = 32768$ ，超出 16 位有符号正数上限 32767，会被解释为 -32768 ，造成符号翻转。因此旋转因子在生成时须限幅到 ± 32767 。

为控制量化误差，本设计对每一级都以 Python 参考模型逐位比对，验证定点实现与浮点参考在允许范围内一致（FFT 相对误差约 0.01%，MFCC 端到端 `diff=0`）。

（五） 运动控制原理

(1) 闭环控制结构。位置闭环的基本结构为：参考位置 r 与实际位置 y 比较得到误差，控制器据此产生控制量 u ，驱动执行器运动，再由位置反馈闭合回路：

$$r \rightarrow e = r - y \rightarrow \text{PID} \rightarrow u \rightarrow \text{虚拟电机} \rightarrow y$$

(2) 五状态状态机。系统用有限状态机管理运动过程。IDLE：系统等待新的控制目标；READY：目标已经输入并等待启动；MOVE：系统执行运动控制；HOLD：机器人到达目标位置并保持；FAULT：检测到故障后停止运动。

目标输入模块负责接收矩阵键盘输入，并对目标角度进行范围检查，将合法目标限制在 $0^\circ \sim 180^\circ$ 范围内。

(3) 限速轨迹规划原理。若把目标位置直接作为 PID 输入，则启动瞬间误差等于整个行程，控制器输出立即饱和，容易造成超调与振荡。轨迹规划器的作用是把“阶跃目标”变为“斜坡参考”，限制执行速度。本设计内部位置用 `deg<<8` 定点表示，每个 1 ms tick 最多移动

$$\Delta = \left\lfloor \frac{VEL_DEG_S \times 256}{1000} \right\rfloor$$

当参考接近目标（距离小于一个步长）时直接置为目标，从而形成梯形或三角形的近似速度曲线。轨迹使能上升沿时把起点重置为当前实际位置，避免起点跳变。

(4) PID 控制器原理。采用离散位置式 PID：

$$e(k) = r(k) - y(k)$$

$$u(k) = K_P e(k) + K_I \sum_{j=0}^k e(j) + K_D [e(k) - e(k-1)]$$

其中比例项（P）按当前误差成比例响应；积分项（I）累加历史误差、消除稳态残差；微分项（D）对误差变化率响应、提供阻尼、抑制超调。数字实现有三个要点：

(1) 积分限幅：积分累加量钳位到 $\pm I_ACC_LIM$ ，防止长时间大误差致积分饱和；

(2) 误差限幅：误差钳位到 $\pm ERR_LIM$ ；

(3) 输出限幅：控制量钳位到 $\pm OUT_LIM$ （对应 ± 60 度/秒），模拟物理上限。

为避免中间截断，三项先左移 PID_SH 位放大，最后一次算术右移。本设计纯比例控制即可收敛，默认 $KP_EFF=2048$ 使 $cmd=2048 \cdot err$ ，对应速度约为 $2 \times$ 误差（度/秒），闭环时间常数约 0.5 s； KI 、 KD 默认 0 ，可按需整定。

(5) 虚拟电机（一阶惯性执行器）原理。用一阶惯性环节模拟真实电机的速度响应：

$$\tau \frac{dv}{dt} = u - v \Rightarrow v(t) = u (1 - e^{-t/\tau})$$

用欧拉法离散化，每个控制 tick 更新一次：

$$v(k+1) = v(k) + \frac{A}{1024} [u(k) - v(k)], \quad A = \left\lfloor \frac{1024}{\tau_{ms}} \right\rfloor$$

$$x(k+1) = x(k) + \frac{v(k+1)}{1024}$$

默认 $TAU_MS=100$ ，即时间常数 0.1 s。位置积分即速度的数值积分，模拟编码器反馈。位置限制在 $0^\circ \sim 180^\circ$ ，到达端点时速度清零，模拟关节机械止挡， at_max/at_min 标志相应置位。

（六） 音频采集与数字接口原理

1. 采样定理与系统时钟

语音信号的能量主要集中在约 300 Hz $\sim$ 3.4 kHz。按奈奎斯特采样定理，采样率须大于信号最高频率的两倍，本设计取 $f_s = 48$ kHz，远高于语音带宽，留有充分裕量。ES8388 需要外部主时钟 $MCLK$ ，其频率通常取采样率的 256 倍：

$$f_{MCLK} = 256f_s = 12.288 \text{ MHz}$$

FPGA 由 PLL 从 50 MHz 产生 12.3077 MHz 的 $MCLK$ ，对应实际采样率约 48.08 kHz，与标称 48 kHz 偏差约 0.16% ，满足要求。

2. I2S 数字音频接口原理

I2S 是面向音频的三线同步串行接口：位时钟 $BCLK$ 、左右声道时钟 LRC ，以及一条串行数据线。本系统中 ES8388 作 I2S 主机，向外提供 $BCLK$ 与 LRC 。位时

钟频率为

$$f_{BCLK} = 2 \times WL \times f_s = 2 \times 24 \times 48k \approx 2.304 \text{ MHz}$$

LRC 的频率等于采样率，其电平与跳变指示当前传输的是左声道还是右声道；数据在 BCLK 沿上按“最高位先行”串行移入。audio_pcm_bridge 在每个 LRC 跳变沿开始接收一个字，移入 24 位得到一个有符号 PCM 采样，并区分左右声道。

本系统的主要音频参数如表 1 所示。

表 1 语音采集系统主要参数

参数	配置
采样率 f_s	48 kHz（实际 ≈ 48.08 kHz）
PCM 数据	24 bit
声道	双声道，默认使用 L 声道
主时钟 MCLK	12.288 MHz（PLL 实际 12.3077 MHz）
位时钟 BCLK	2.304 MHz
接口	I2C + I2S
音频编解码器	ES8388

3. I2C 配置接口原理

ES8388 的内部寄存器通过 I2C 总线配置。一次写操作的时序为：

START 起始条件 → 7 位从机地址与写位 → 从机 ACK → 8 位寄存器地址 → ACK → 8 位数据 → ACK → STOP 停止条件。

本系统实测该模块的从机地址为 0x11（7 位地址 0010001）。es8388_config 把上电所需的寄存器配置组织为一张表，由 i2c_reg_cfg 逐条下发，i2c_dri 负责 SCL/SDA 的位级时序与 ACK 检测。采样率、位宽、输入选择、PGA 增益等音频参数均通过该链路写入。

4. 跨时钟域采集

如（三）其四所述：aud_bclk 域的串行接收与 sys_clk 域的后续处理之间，用翻转电平握手配合两级触发器同步完成，保证多比特 PCM 数据在跨域采样时已经稳定。

(七) 语音活动检测 (VAD) 原理

VAD 用于把连续音频流中的有效语音段与静音、噪声分离，是分段处理的前提。本设计以每 256 个立体声采样（约 5.3 ms）为一帧，取帧内左右声道绝对值的最大值

$$p = \max_n \max(|L[n]|, |R[n]|)$$

作为帧特征。判决策略采用双阈值滞回：当某帧峰值 $p \geq TH_{ON}$ 时进入语音态；进入语音后，只有当 $p < TH_{OFF}$ 且连续 HO_FRAMES 帧都为低（拖尾 hangover）时才判定语音结束。滞回避免了在阈值附近反复抖动；拖尾参数（32 帧 ≈ 170 ms）保证词内短暂停顿不会把一句话切碎。 $TH_{ON} > TH_{OFF}$ ，两者均由板级实测数据定标。

(八) 语音特征提取原理 (MFCC)

Mel 频率倒谱系数 (MFCC) 模拟人耳对频率的非线性感知，是语音识别与声纹识别中广泛使用的特征。整体处理流程为：

$$\begin{aligned} & \text{PCM} \rightarrow \text{VAD} \rightarrow \text{预加重} \rightarrow \text{分帧加窗} \\ & \rightarrow \text{FFT} \rightarrow \text{Mel 滤波} \rightarrow \text{Log} \rightarrow \text{DCT-II} \rightarrow \text{MFCC} \end{aligned}$$

各环节的原理如下。

1. 预加重

语音的功率谱随频率约按 -6 dB/倍频程下降。预加重用一阶高通滤波提升高频，使频谱更平坦、便于统一分析：

$$y[n] = x[n] - a x[n-1], \quad a \approx 0.97$$

定点实现取 Q14，即 $a = 15892/2^{14} \approx 0.9697$ 。

2. 分帧与加窗

语音是短时平稳信号，在 $10 \sim 30$ ms 尺度上可近似认为其统计特性不变。本设计帧长 $N = 64$ （48 kHz 下约 1.33 ms），帧长较短以降低运算量、提高时间分辨率，帧间无重叠（hop=N）。对每帧加 Hann 窗：

$$w[n] = 0.5 \left(1 - \cos \frac{2\pi n}{N-1} \right)$$

加窗的作用是减小帧两端因截断造成的旁瓣的频谱泄漏，使 FFT 结果更接近真实频谱。窗系数以 Q15 定点表示。

3. 快速傅里叶变换 (FFT)

对每帧做 N 点离散傅里叶变换, 把时域信号变换到频域:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad W_N = e^{-j2\pi/N}$$

直接计算复杂度为 $O(N^2)$ 。本设计采用基-2 按时间抽取 (DIT) 算法, 利用旋转因子的周期性与对称性把 N 点 DFT 递归分解为两个 $N/2$ 点 DFT, 基本蝶形运算为

$$X'_a = X_a + W X_b, \quad X'_b = X_a - W X_b$$

其中复数乘法由实部、虚部组合实现:

$$\Re(W X_b) = W_r \Re(X_b) - W_i \Im(X_b), \quad \Im(W X_b) = W_r \Im(X_b) + W_i \Re(X_b)$$

算法共 $\log_2 N = 6$ 级、每级 $N/2 = 32$ 个蝶形, 复数乘法次数由 $O(N^2)$ 降到 $(N/2) \log_2 N$ 。DIT 要求输入为位倒序、输出为自然频率序, 本设计在装载时按位倒序写入。旋转因子以 Q15 定点常量给出。为防止逐级运算溢出, 每个蝶形输出右移 1 位 (等效于整体除以 N)。FFT 缓存使用块 RAM (BRAM) IP, 相比寄存器阵列大幅降低了 LUT 占用。

4. 功率谱

对每个频点取模平方:

$$P[k] = \Re(X[k])^2 + \Im(X[k])^2$$

得到功率谱, 作为 Mel 滤波器组的输入。为适应定点位宽, 整体右移 PS=22 位定标。

5. Mel 滤波器组

人耳对频率的感知近似对数关系, Mel 刻度描述这一特性:

$$\text{mel}(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

在 Mel 域上均匀布置 $M = 20$ 个三角滤波器, 映射回线性频率后对功率谱加权求和, 得到各子带能量:

$$\text{mel}[m] = \sum_k P[k] c_m[k]$$

三角窗系数以 Q12 定点存储。Mel 滤波器组既实现降维，又保留了语音的感知相关信息，同时压缩了与识别目标无关的细节。

6. 对数运算

对每个子带能量取对数，压缩动态范围，并把乘性关系变为加性：

$$L[m] = \log(\text{mel}[m])$$

硬件中以 $\log_2$ 近似实现：先用位扫描求出最高有效位的位置，再取尾数高 6 位查表得到小数部分，合成为 $L = e \cdot 2^{FW} + \text{LUT}(idx)$ 。

7. 离散余弦变换 (DCT-II) 与 MFCC

对 M 个对数 Mel 系数作正交 DCT-II，取前 $K = 13$ 个系数：

$$y[k] = c_k \sum_{m=0}^{M-1} L[m] \cos \left[\frac{\pi}{M} \left(m + \frac{1}{2} \right) k \right], \quad c_0 = \sqrt{1/M}, \quad c_{k>0} = \sqrt{2/M}$$

DCT 的作用是去相关与能量压缩——把相邻子带间高度相关的能量集中在少数低阶系数上，从而用 13 维即可代表整帧的频谱包络。最终输出的 13 维向量即 MFCC 特征，供声纹认证与命令识别两条链共用。

本设计主要参数为：FFT 点数 $N = 64$ ，hop=64，Mel 滤波器数量 $M = 20$ ，MFCC 维数 $K = 13$ ，预加重系数 0.97，Hann 窗与旋转因子采用 Q15 定点。为适应 FPGA 定点运算，系统对中间计算结果做统一位宽管理，并采用饱和策略避免溢出。

(九) 声纹认证与段级决策原理

声纹认证属于 Who 处理链，其目标是判断当前说话人是否为主人。

1. 模板匹配与 L1 距离

系统预先存储主人多段语音的 MFCC 均值模板 $\{t_d\}$ 。对一段语音，计算其 MFCC 与模板的 L1 距离：

$$D = \sum_{d=1}^{13} |x_d - t_d|$$

选用 L1 而非 L2 的原因是，1 不含乘法，硬件代价低；且对个别维度的异常值更鲁棒。距离小于阈值即判为匹配。

2. 从逐帧决策到段级决策

最初实现是对每帧 MFCC 单独判距离并投票。板级实测发现：主人的帧距离与陌生人的帧距离分布几乎完全重叠（约 3584~7424），逐帧无法用单一阈值分开。原

因是单帧（1.33 ms）信息量太小、随机波动大。改为段级决策后，把 VAD 语音段内所有帧的 MFCC 求和，段末用段均值判断：

$$\bar{x}_d = \frac{1}{cnt} \sum_i x_{id}, \quad D_{seg} = \sum_d |\bar{x}_d - t_d| = \frac{1}{cnt} \sum_d \left| \sum_i x_{id} - cnt \cdot t_d \right|$$

平均抑制了随机波动，主人的段均值距离（约 265~4106）与陌生人的（约 4486~9134）清晰分离。这一决策由 `seg_decide` 模块实现。

3. 无除法实现

FPGA 中除法代价高。注意到判决只需比较 D_{seg} 与阈值，因此我把上式乘以 cnt 消去除法，只做整数乘加与比较：

$$\sum_d \left| \sum_i x_{id} - cnt \cdot t_d \right| < cnt \cdot TH$$

段末状态机在 13 个维度上顺序累加绝对值，再用一个 32 步恢复除法器单独计算精确段均值，仅用于调试显示。

4. 阈值数据定标

阈值不是凭经验设定，而是由板上真实采集数据确定的。在主人的段均值距离上界与陌生人的下界之间取定。当前取 $TH_OWN=4300$ ，落在主人最大 4106 与陌生人最小 4486 之间。此外，PC 端引入 CAM++ 作为离线参考模型，用等错误率（EER）与 leave-one-out 方法检验特征可分性。

5. 跨会话声学失配

实测发现一个值得记录的现象。若建立模板时的录音条件（距离、音量、位置）与测试时不同，同一词语的 MFCC 会有显著差异，能量主导的前几维甚至出现符号翻转，导致模板失效。

这说明基于 MFCC 模板的方法只对采集时的声学条件有效。工程上的应对是同条件采集与测试，长期则需引入能量归一化或更鲁棒的特征。

（十） 命令识别原理

命令识别属于 What 处理链，判断“说了什么”。本设计采用四类命令模板：

stop, left, right, forward

每个命令建立对应 MFCC 模板，识别时计算输入段 MFCC 均值到各模板的 L1 距离，取距离最小者作为判决：

$$\hat{c} = \arg \min_{c \in \{0,1,2,3\}} \sum_d \left| \bar{x}_d - t_d^{(c)} \right|$$

该判决与声纹判决共享同一段累加结果 (`seg_decide`)。命令编号与运动映射如表 2 所示。

表 2 语音命令编号及运动映射

命令	cmd_id	运动目标
stop	0	停止/保持
left	1	0°
right	2	180°
forward	3	90°

实测局限如下：以 13 维 MFCC 和为特征时，stop/left/right 三者的谱形非常接近（去掉能量维后的距离只有几百），仅 forward 可分，左、右互相混淆是主要误差来源。这是特征层的能力限制，而非实现错误，后续可通过更鲁棒的特征或更深模型改善。

（十一） 双麦克风方向感知

系统用两颗麦克风分别接入左右声道，通过比较段内能量估计声源方向。在 VAD 语音段内逐采样累加左右声道的幅度：

$$S_L = \sum_n |L[n]|, \quad S_R = \sum_n |R[n]|$$

段末按比例判决。为避免除法，采用交叉相乘比较： $S_L \cdot DEN > S_R \cdot NUM \Rightarrow$ 左； $S_R \cdot DEN > S_L \cdot NUM \Rightarrow$ 右；否则 $\Rightarrow$ 中

其中 NUM/DEN 决定“偏侧”门限（本设计取 1.05，即左右能量比超过约 5% 即判为偏侧）。交叉相乘使判据与绝对能量无关，因而对音量变化不敏感。方向结果与命令联合：当主人说“前进”时，按声源方向执行左转（目标 0°）、右转（目标 180°）或前进（目标 90°）。

(十二) Who + What 联合决策模块

联合决策模块是 V2 的核心。系统只有在

$$owner_valid \wedge cmd_valid$$

同时成立时才执行对应命令。如果说话人不是主人，即使 What 链识别出了有效命令，也不会改变机器人目标。这种结构可以有效避免陌生人通过语音直接控制机器人。

两个条件都在 VAD 语音段结束时由 `seg_decide` 给出：`owner_valid` 为段内锁存电平，`cmd_id` 为段末锁存结果。`decision_fsm` 将命令编号映射为动作（0 停止，2 前进，3 左转，4 右转），并只在主人身份成立时才把动作送入运动链。特别地，停止命令（action 0）会令目标等于当前位置，实现“立即保持”。

(十三) PC 数字孪生模块

PC 端包括 Python 工具链和 Web 数字孪生系统。

Web 数字孪生采用 HTML、CSS、JavaScript 和 Web Serial API 构建，无需额外框架。系统可以显示虚拟机械臂、状态机、目标位置、实际位置、误差、速度以及故障事件。FPGA 通过 UART 以 115200 baud、约 10 Hz 发送固定格式遥测帧

[AA][state][target][pos][chk]

其中第 2 字节为状态机编号（0~4 对应 IDLE~FAULT），`chk` 为前面各字节的异或校验。浏览器端解析该帧并驱动 SVG 机械臂与滚动曲线。

数字孪生提供 Demo 和 Live 两种模式：Demo 模式内置与 RTL 同构的轨迹限速、PID 与一阶电机模型，用于无 FPGA 条件下测试控制逻辑；Live 模式则通过 Web Serial 接收 FPGA 实际遥测数据。语音与键盘是并行的两个输入源，最终都汇入同一运动链。

三、 理论设计仿真电路

(一) 仿真电路

本设计采用 ModelSim 对 Verilog 模块进行仿真验证。系统建立 Python Golden 参考模型，由 Python 产生标准输入和期望输出，再由 Testbench 逐项比较 RTL 输出。

整体验证过程为：

Python Golden → .mem 测试向量 → RTL → ModelSim → PASS/FAIL

整个仿真环境的组成为：左侧用 Python 建立与 RTL 逐位一致的整数镜像参考模型(audio_golden.py 与各 gen_*.py),生成 .mem 测试向量与期望输出;中部 ModelSim 由 Testbench 施加时钟、复位与激励,被测设计为整链顶层 top_voice_robot;右侧 Testbench 将 RTL 输出与 Golden 期望逐位比较,输出 PASS/FAIL。此外,板上 ES8388 采集的真实语音经 UART 遥测回传 PC 离线复算,用于定标段级阈值并参与逐位比对。

除 Testbench 外,声纹认证、命令识别与联合决策三部分还进一步用板上真实采集的语音段数据(ES8388 采集后由 FPGA 输出各段 13 维 MFCC 段内和)进行离线复算与绘图,使波形反映的是真实数据而非合成激励。下文图 3、图 4、图 7 均基于该真实采集数据集绘出。

1. V1 运动控制闭环仿真

tb_top_system 对目标输入、状态机、轨迹规划、PID、虚拟电机和故障检测进行整体仿真。

给定目标位置后,轨迹规划模块产生平滑参考轨迹,PID 控制器根据参考位置 and 实际位置之间的误差产生控制量,虚拟电机随后更新实际位置。

运动控制闭环的数据通路为:目标源(键盘或语音)给出 target/go 后, state_machine 完成状态转移(IDLE2READY2MOVE2HOLD,任意态故障进 FAULT); trajectory_planner 以 $\text{deg} \ll 8$ 定点、每控制 tick 最多移动 $\text{STEP} \approx 38$ (约 0.15°) 逐步逼近目标,输出平滑参考位置 ref_pos; pid_controller 对位置误差取限幅(± 180)后经 $K_P=2048$ 放大、三项合成右移 10 位,输出 ± 61440 限幅的速度指令; virtual_motor 以 $\text{TAU}=100\text{ms}$ 的一阶惯性逼近指令速度并对位置积分,反馈 pos 与目标比较(偏差 ≤ 2)得到 at_target 回送状态机,构成完整闭环。

仿真结果表明,实际位置能够跟随参考轨迹逐渐接近目标位置,并在到位后进入 HOLD 状态。

2. ES8388 与 I2S 数字接口仿真

ES8388 配置模块通过 I2C 完成寄存器写入,音频数据通过 I2S 接口进入 FPGA。

仿真重点验证:I2C 起始、地址、寄存器地址和数据传输,I2S BCLK 与 LRCK 时序,左右声道数据采样,24 bit PCM 数据组帧,音频数据有效信号生成。

音频采集电路的 RTL 结构为:PLL 输出约 12.3MHz 的 MCLK 供给 ES8388,I2C 将其 ADC 输入配置为 IN1(咪头);codec 侧产生 BCLK 与 LRCK 后,

audio_pcm_bridge 在 aud_bclk 域按 LRC 沿判断字起始、MSB 先行收满 24 bit，第 2/4...个字提交左右声道缓冲，并以 pair_tgl 翻转握手跨入 50MHz 主域；主域经 2 级触发器同步后作沿检测，得到约 48kHz 的 pair_valid，同时锁存 pcm_l/pcm_r。该结构把异步 I2S 输入安全转换为同步单时钟域信号，后续 VAD 用窗能量加双阈值滞回产生 vad_on。

3. MFCC 特征提取仿真

MFCC 前端采用模块化流水线。

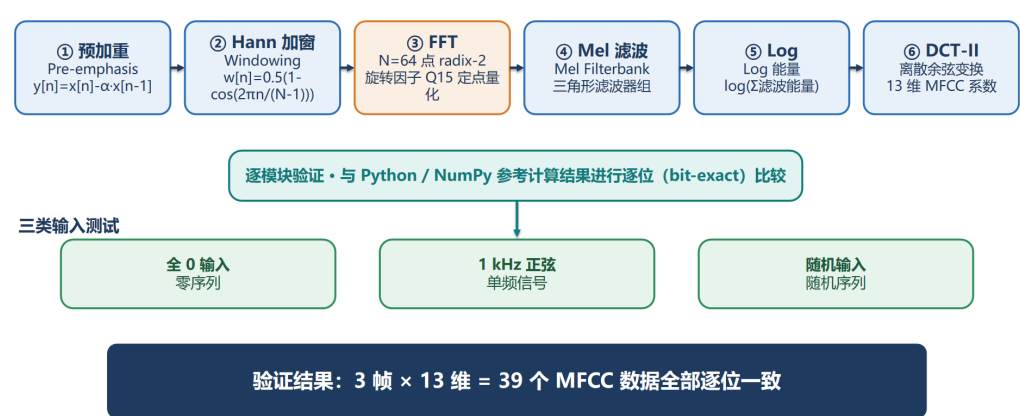


图 2 MFCC 特征提取流水线

预加重、Hann 加窗、FFT、Mel 滤波、Log 和 DCT-II 分别进行验证。

FFT 采用 $N = 64$ 点 radix-2 结构，并对旋转因子进行 Q15 定点量化。测试结果与 Python/NumPy 参考计算结果进行比较。

在全 0、1 kHz 正弦以及随机输入三类测试中，3 帧共 39 个 MFCC 数据均完成逐位一致性验证。

MFCC 特征提取的 RTL 数据通路与图 2 的算法流程对应：24 bit PCM 依次经过预加重（Q14， $\alpha \approx 0.97$ ）、Q15 Hann 加窗、64 点 radix-2 DIT FFT（旋转因子 Q15 量化、块 RAM 缓存蝶形中间量）、功率谱（ re^2+im^2 右移 22 位）、20 路 Mel 三角滤波（Q12）、log2 查表与正交 DCT-II（基矩阵 Q12），输出 13 维 int16 系数；内部以 win、powr、lm20 三块 RAM 完成帧间缓存，feature_engine 的状态机统一调度各级流水。

4. 声纹认证仿真

tb_speaker_verify 用于验证单帧 MFCC 与主人模板之间的距离计算；

`tb_utter_vote` 用于验证多个片段的投票逻辑。在全命中、全不命中、临界值以及边界条件等 7 类场景下，仿真结果均符合预期。

在此基础上，针对逐帧不可分的问题，新增段均值决策仿真：把 VAD 段内各帧 MFCC 累加，段末以“不除法”等价式判决主人身份与命令类别。对应测试台 `tb_seg_decide` 的 6 类场景全部通过。

段级决策电路的结构为：`feature_engine` 每帧输出 13 个系数，段内（仅 vad 期间）累加器维护 `sum[0..12]` 与帧计数 `cnt`；段末 FSM 逐维并行计算 5 个模板的距离累加 `acc_v` 与 `acc0..3`（模板以参数常量内嵌），最终以无除法等价式判定主人： $acc_v < cnt \cdot TH_OWN$ （`TH_OWN=4300`），并对 `stop/left/right/forward` 四个模板取 `argmin` 得到 `cmd_id`。

逐帧 `speaker_verify` 及 `utter_vote` 因对照路径因帧级距离分布重叠（约 3584~7424）不可分而弃用。

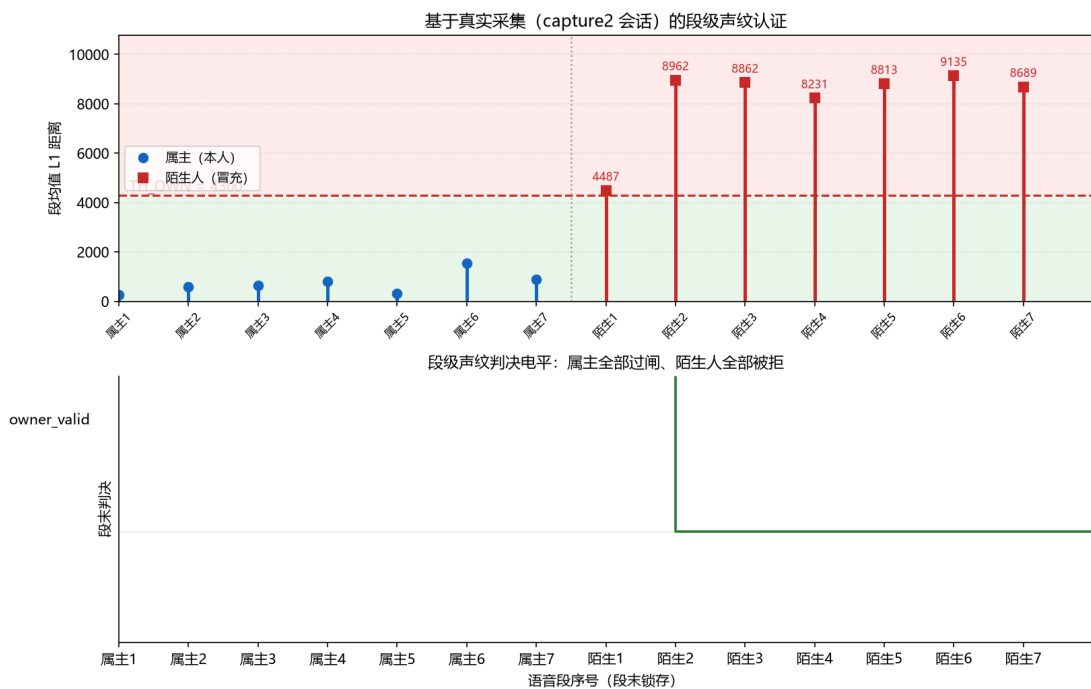


图 3 声纹认证与段级决策波形（基于板上真实采集数据）

5. 命令识别仿真

`tb_cmd_matcher` 对 `stop`、`left`、`right`、`forward` 四类模板进行距离比较，`tb_cmd_vote` 对连续识别结果进行窗口投票。

逐帧命令识别电路的结构为：4 个 `vtmpl` 模块（`stop/left/right/forward` 模板）并行对逐帧到来的 13 维特征累加 L1 距离，比较树以最小距离对应的模板索引作为

本帧命令，随后 cmd_vote 用 8 帧滑动窗口取众数输出稳定判决。

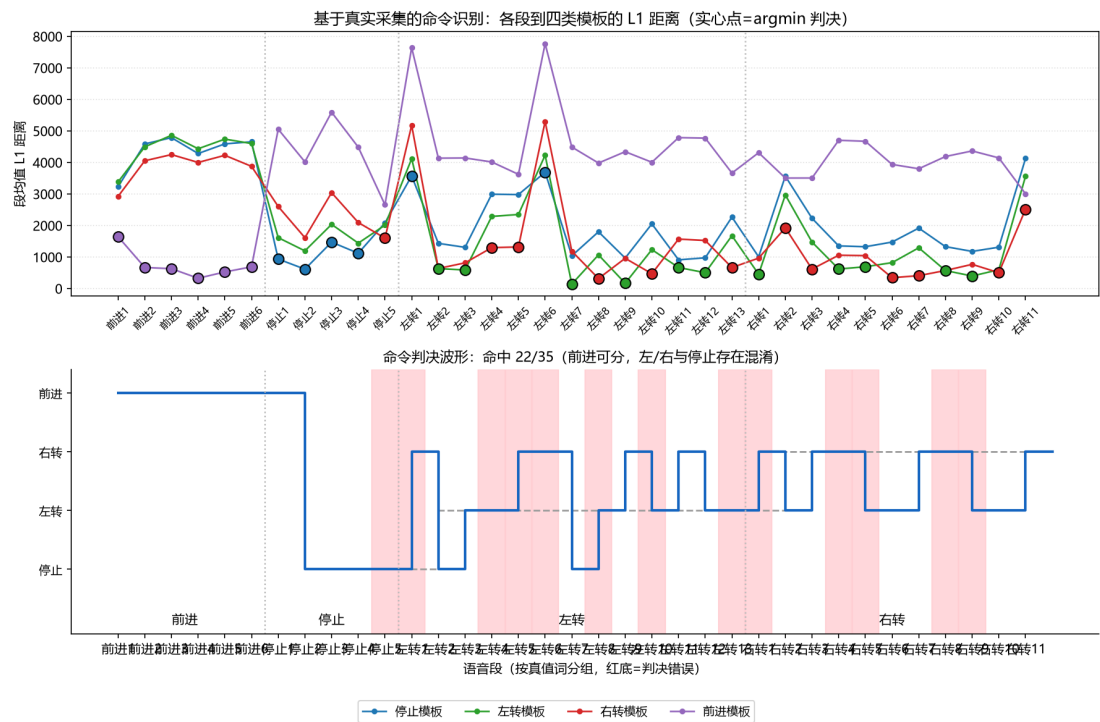


图 4 KWS 命令识别波形（基于板上真实采集数据）

6. Who 及 What 联合决策仿真

联合决策通过 tb_voice_control 和 tb_top_voice_system 验证。

主要测试三个场景：

- 1. 主人说 left：身份认证通过，命令有效，机器人目标变为 0°；
- 2. 陌生人说话：即使命令有效，也不执行动作；
- 3. 主人说 stop：系统进入停止或保持状态。

三种场景均通过仿真验证。

7. 双麦方向与回归仿真

tb_dir_energy 实现比较两麦克风输入能量的差异来分辨方向。在左右能量占比超过门限时分别判为左、右，否则为中，含 1.25 倍边界条件共 7 类场景全部通过。此外，tb_top_voice_robot、tb_voice_dbg、tb_cap_rpt、tb_cmd_lit 等顶层测试台用于联合回归，确保语音链接入后 V1 键盘运动与 [AA] 遥测不退化。

双麦方向感知与联合决策电路的结构为：`dir_energy` 在 VAD 段内累加左右声道幅度，段末以交叉相乘（ $SL \cdot 21$ 与 $SR \cdot 20$ 比较）避免除法，能量比超过 1.05 倍才判偏侧；`decision_fsm` 接收段级 `owner` 与 `cmd` 结果，`owner` 通过且命令有效时译出动作（0 停/2 前/3 左/4 右），顶层 `seam` 仅在命令为“前进”时叠加方向，把目标置为左/右/前对应角度。

（二） 运行结果

1. Python Golden 与 RTL 逐位验证

用 Python 建立与 RTL 逐位一致的整数镜像模型 `audio_golden.py`，对真实录音（48 kHz 单声道 PCM，非重叠 64 采样分帧、能量门取有效帧）同时计算 Golden 特征，并把同一批帧送入 ModelSim 中的 RTL `feature_engine`，逐帧逐维比对。以一段真实主人录音为例，取 16 个有效帧，共 $16 \times 13 = 208$ 个 MFCC 系数，RTL 与 Golden 完全一致（ $\max|\text{diff}|=0$ ）。

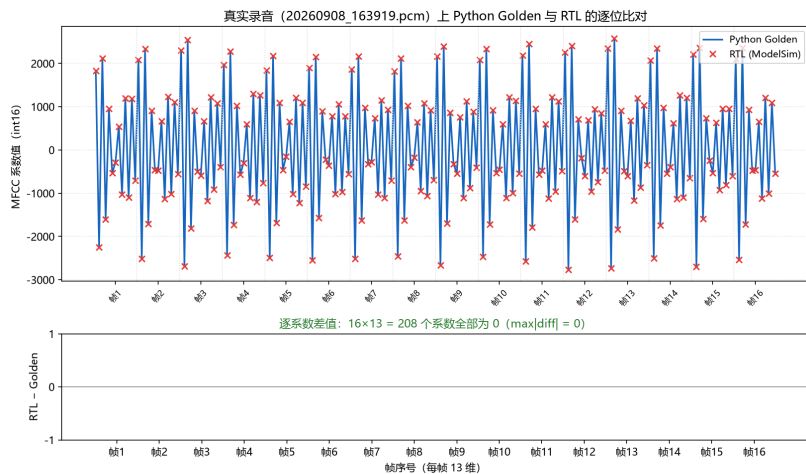


图 5 Python Golden 与 RTL 逐位比对（真实录音，16 帧 $\times$ 13 维全部 $\text{diff}=0$ ）

该验证方式能够在 RTL 上板之前发现定点位宽、符号和量化误差问题，提高硬件实现可靠性。

2. 声纹实验结果

使用 15 条主人录音和 24 条冒认录音，对 MFCC13+L1 基线和 CAM++ 进行离线比较。

结果如下表。评估工具最初输出的数值实为 ROC 曲线与对角线相交处的最小 FAR 与 FRR 之差；同时按定义重新计算了真正的等错误率 EER，两者一并列出。

表 3 声纹离线评估结果（真实录音）

指标	full		leave-one-out	
	MFCC13+L1	CAM++	MFCC13+L1	CAM++
最小 FAR/FRR 之差	0.000	0.017	0.000	0.000
等错误率 EER	0.300	0.054	0.333	0.054

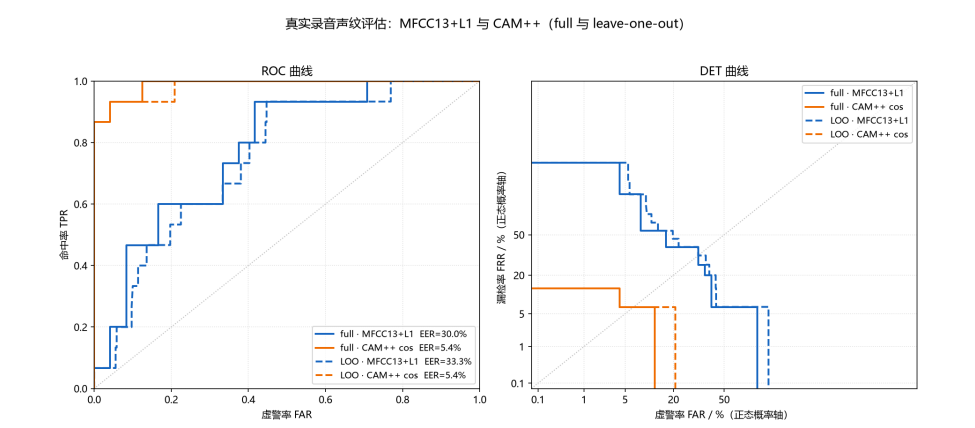


图 6 声纹评估 ROC 与 DET 曲线（真实录音：MFCC13+L1 与 CAM++，full 与 leave-one-out）

由于当前数据量较小，且主人与冒认录音之间存在较高的内容相似性，因此上述结果不能直接代表真实环境下的泛化性能。特别是 Full 模式模板包含测试身份数据，因此存在一定乐观偏差。后续仍需要增加独立录音和不同说话内容进行验证。从真实 EER 看，在当前数据上 MFCC13+L1 的判别力有限（约 30%），而 CAM++ 明显更好（约 5%），这说明在资源受限的轻量级特征下，单条录音级模板匹配不足以保证可靠区分说话人，也正是后文采用段级决策的动因。

进一步地，用 ES8388 在板上采集真实语音后重新评估发现：逐帧 MFCC 的主人与陌生人距离分布几乎完全重叠（约 3584~7424），单帧不可分；而改用段均值后，主人段均值距离约为 265~4106，陌生人为 4486~9134，明显分离。据此把声纹阈值定为 TH_OWN=4300，落在两者之间。这一结果同时说明：在资源受限的轻量级实现中，合理的决策粒度（段级和逐帧比较）对系统可用性的影响甚至超过特征本身。

3. 双引擎端到端仿真

三个测试场景均 PASS，证明身份认证结果能够正确约束命令执行路径。

在此基础上，系统完成了完整的板级闭环验证：主板对麦克风说“前进”，FPGA 内部依次完成 I2S 采集、VAD 分段、MFCC 特征提取、段级声纹认证与命令识别；

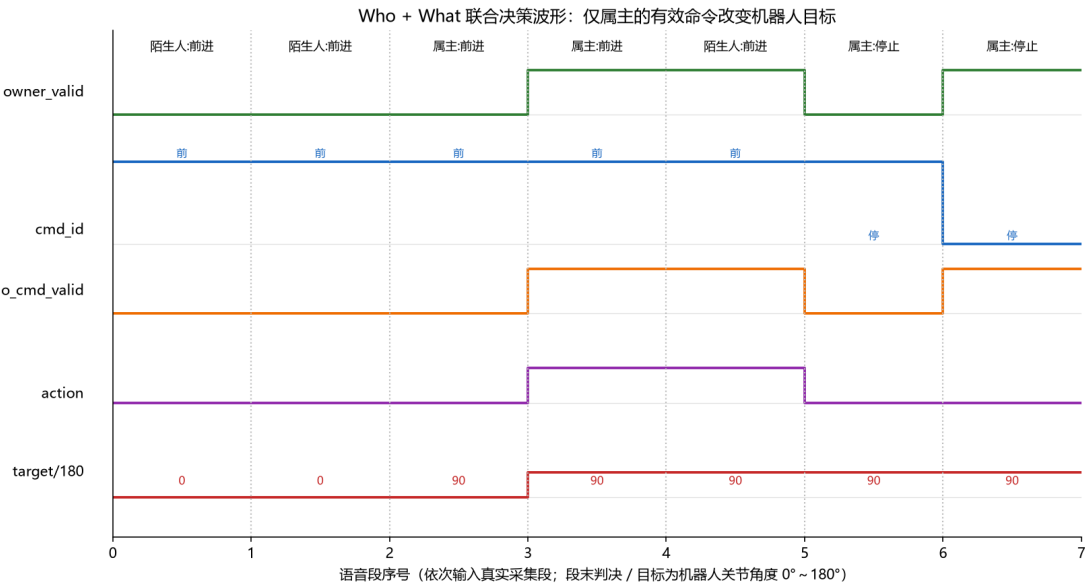


图 7 基于板上真实采集数据的 Who + What 端到端联合决策波形

联合决策输出动作，映射为目标 90°；随后轨迹规划与 PID 控制驱动虚拟电机运动；同时 UART 以 [AA] 帧持续上报，PC 数字孪生机械臂从当前位置平滑转到 90°，状态机同步经历 IDLE→READY→MOVE→HOLD。

4. 数字孪生运行结果

如图 8所示数字孪生可以实时显示目标位置、实际位置、状态机以及运动曲线，并能够通过 UART 接收 FPGA 实际运行状态。

(三) 电路设计亮点、存在问题及改进方法

1. 电路设计亮点

本设计主要具有以下特点：采用单一系统主时钟配合 clock-enable 的时钟设计方式；采用同步复位和严格的时序、组合逻辑划分；音频采集跨时钟域用翻转电平握手和两级触发器同步处理，兼顾可靠性与实现简洁；定点计算采用统一位宽管理、饱和与三限幅策略；FFT 缓存由寄存器阵列改为块 RAM，显著降低 LUT 占用；感知、决策和执行模块相互解耦，键盘与语音为并行输入源、共用同一运动链；Who 和 What 两条语音处理链共享 MFCC 特征但独立验证；采用段均值决策替代逐帧决策，显著改善在有限特征下的判别稳定性；Python Golden 与 RTL 逐位验证提高了算法硬件化的可靠性；数字孪生提高了系统内部状态的可观察性。

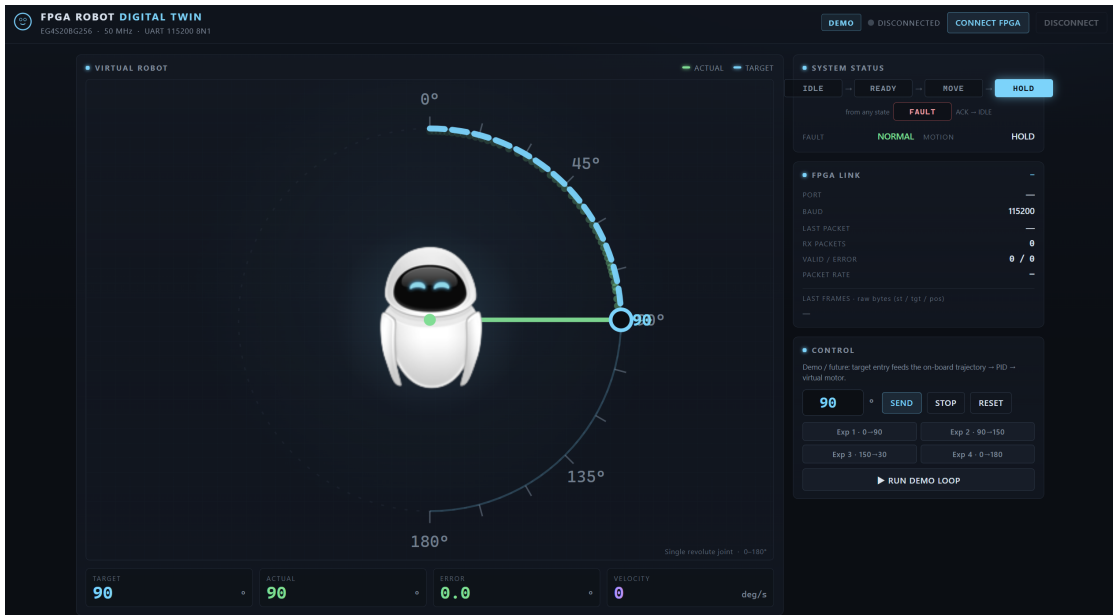


图 8 Web 数字孪生系统界面

2. 存在问题及改进方法

首先，13 维 MFCC 和特征对命令词的判别能力有限，stop/left/right 谱形接近，左、右互相混淆。后续可采用更鲁棒的特征（如加入差分系数、能量归一化）或量化后的小型神经网络模型改善。

其次，基于模板的方法只在“采集时的声学条件”下有效，不同距离、音量、位置会出现声学失配。后续需要引入能量归一化，或强制同条件采集与测试。

再次，V1 自动堵转检测在低速和整数角度分辨率条件下存在误判，因此当前整机演示采用 SW2 手动触发故障。后续可以提高位置和速度分辨率，并加入滞回和持续时间判断。

最后，声纹阈值目前由实测数据在主人与陌生人距离之间取定，数据量仍然有限，后续应扩大数据集并纳入更多说话人。

四、 硬件电路设计

（一） FPGA 实验板硬件资源分配及使用情况

本设计采用安路 EG4S20BG256 FPGA 开发板。

主要硬件资源包括：19600 LUT、193 个可用 IO、1 Mbit BRAM 及分布式存储资源、4 个 PLL、50 MHz 板载晶振、12 个 LED、4 位共阴数码管、4×4 矩阵键盘、独立按键及 8 个拨动开关、CH340 UART 转 USB、ES8388 音频模块。

V1 系统主要引脚分配如下。

表 4 V1 系统主要 FPGA 引脚分配

信号	FPGA 引脚	功能
clk	R7	50 MHz 系统时钟
rst_n	A9	SW0 复位输入
row[3:0]	D9、F9、C10、E10	键盘行
col[3:0]	F10、C11、D11、E11	键盘列
fault_sw	B10	手动故障输入
tx	D12	UART 发送
seg[7:0]	A4、A6、B8、E8、A7、B5、 A8、C8	数码管段码
dig_cs[3:0]	C9、B6、A5、A3	数码管位选
led[7:0]	B14、B15、B16、C15、C16、 E13、E16、F16	状态指示

ES8388 接口主要引脚如下。

表 5 ES8388 主要接口引脚

信号	FPGA 引脚	功能
aud_mclk	N5	音频主时钟
aud_bclk	M6	I2S 位时钟
aud_lrc	M7	I2S 左右声道时钟
aud_adcdat	R14	ADC 数字输出
aud_dacdat	P6	DAC 数字输入
aud_scl	R12	I2C 时钟
aud_sda	R9	I2C 数据

最终统一顶层 top_voice_robot 在 TD 工具中综合后的资源占用如表 6 所示，整体在芯片容量之内；其中把 FFT 缓存由寄存器阵列改为块 RAM（BRAM）IP 是降低 LUT 占用的关键优化。

表 6 top_voice_robot 综合资源占用

资源	占用	说明
LUT	约 6440 / 19600 (32.9%)	约占总逻辑资源三分之一
触发器 FF	约 1669	
— DSP	28 / 29	FFT、Mel、DCT 乘加密集
BRAM	4	FFT 缓存 + 系数模板表
PLL	1	音频主时钟

(二) FPGA 外围电路原理图

本节依据《EG4S20BG256 核心板》原理图，绘制本设计所使用 FPGA 核心板的外围电路。各图均采用真实元件位号、FPGA 引脚编号与网络名称，元件符号采用国标（GB）画法。

1. FPGA 供电电路

核心板以 5V 供电：输入先经保险丝 F1（SMD1812P050TF），D9（SS54）以阴极接 5V、阳极接 GND 的方式作为反接钳位保护，R81（2.21k）串接 POWER2 红色 LED 作电源指示。

两路同步降压均采用 RT8097CHGB 五脚器件：U5 经 L1（SWPA6028S4R7MT）产生 FPGA 内核 1.2V（VCCINT），U6 经 L3 产生 FPGA I/O 3.3V；两路输出分别配 22μF 与 4.7μF 去耦电容，并以 R83/R84（10.0k）与 R85（10.0k）/R86（49.9k）分压回馈。

L4（120Ω）把 FPGA3.3V 与板级 3.3V 隔开，两侧分别配 C21（4.7μF）与 C22（0.1μF）。5V 另经 U11（LD1117-3.3）产生外设轨 OUT_3.3V，输入配 C55（4.7μF）、输出配 C15（4.7μF）与 C16（0.1μF）。

如图 9 所示。

2. FPGA 配置电路

FPGA 采用主动串行（AS）配置：SPI Flash M25P16（U8）的 CS、CLK、DI/IO0、DO/IO1 分别经 SPI_CSN、FPGA_CCLK、FPGA_MOSI、FPGA_D0 四线接入 FPGA 配置 Bank（U7E，CSO_B=T3、CCLK=R11、MOSI=T10、D0/DIN=P10）；WP/IO2 与 HOLD/IO3 各经 R95/R96（2.21k）上拉到 FPGA3.3V，VCC 由 C47（0.1μF）退耦。

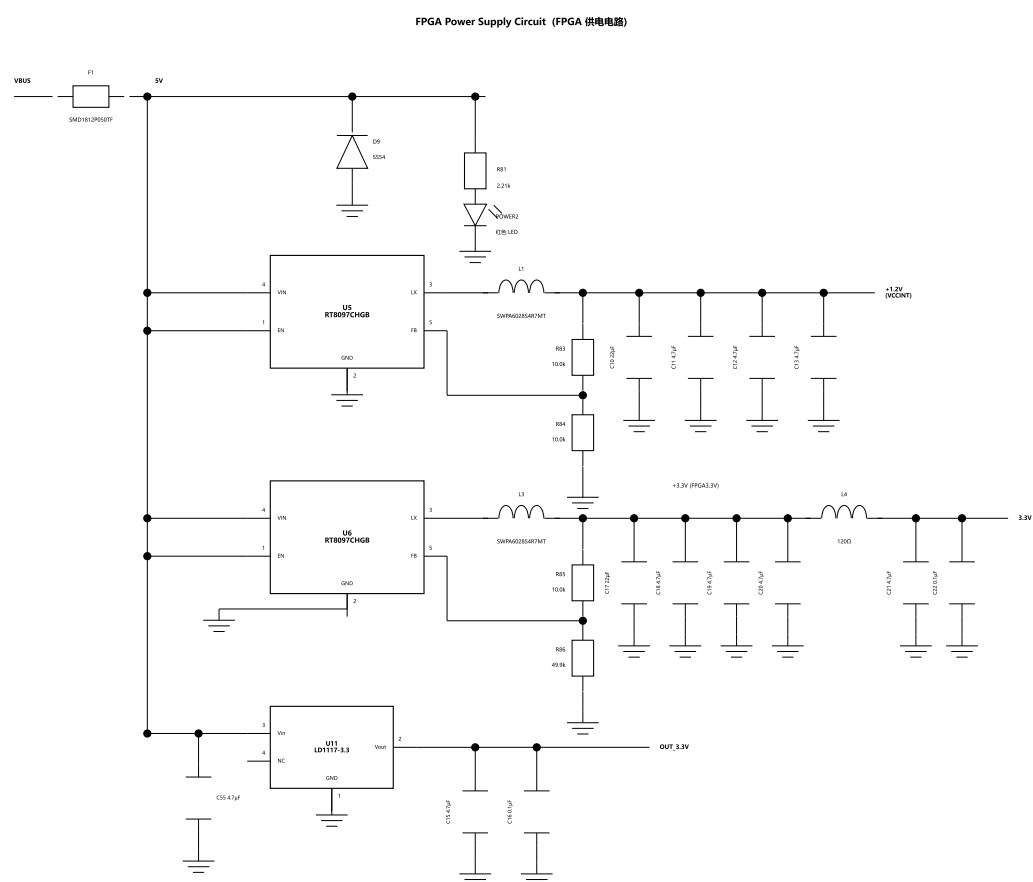


图 9 FPGA 供电电路

配置状态 INIT_B (R3) 、 DONE (P13) 与 PROGRAM_B (T2) 分别由 R89/R90/R91 (10.0k) 上拉；模式选择 M0 (T11) 经 R92 (470Ω) 上拉、M1 (N11) 经 R93 (470Ω) 下拉。

JTAG 侧 TCK (C14) 、 TDO (E14) 、 TMS (A15) 、 TDI (C12) 各经 R77/R78/R76/R79 (0Ω) 连到 USB_JTAG_TCK/TDO/TMS/TDI 网络。

如图 10 所示。

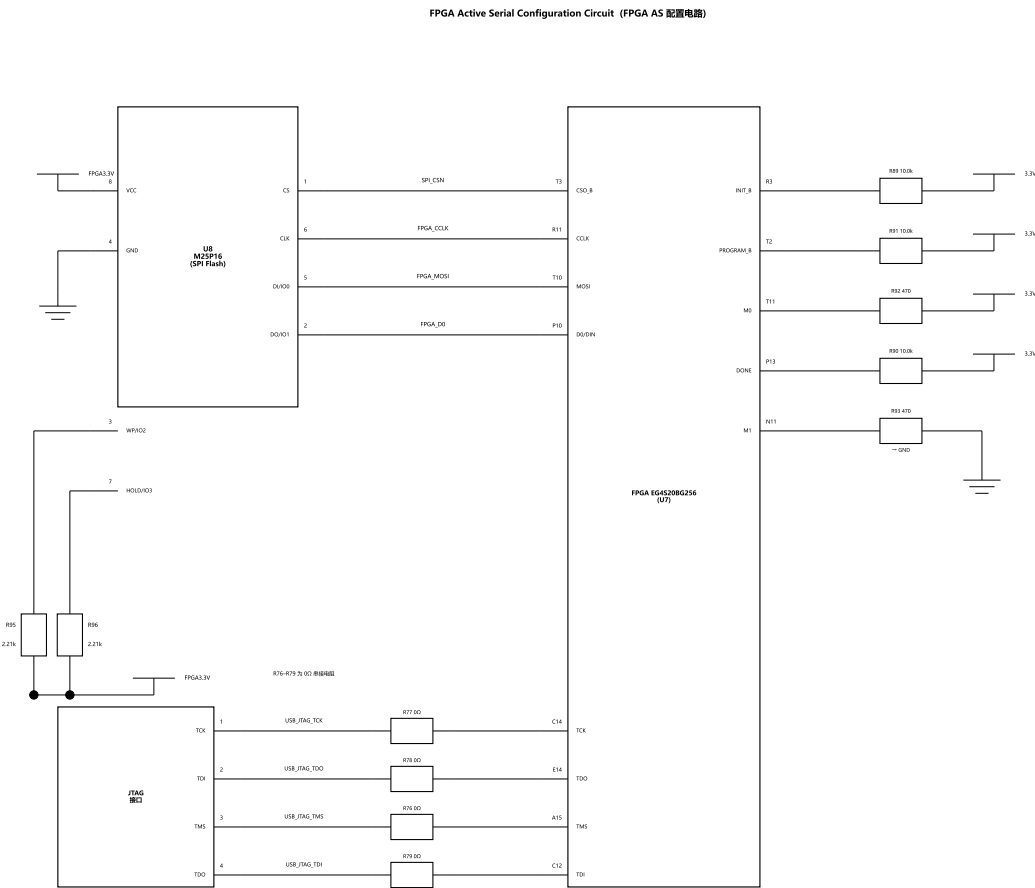


图 10 FPGA AS 配置电路

3. FPGA 时钟电路

系统时钟由 50MHz 有源晶振 Y3 (7X-50.000MBB-T) 提供：VDD 接 3.3V 并配 C35 旁路，OUT 经 R97 (22.1Ω) 串阻接 FPGA 全局时钟引脚 FPGA_GCLK1 (T8)。如图 11 所示。

4. 外设接口电路

外设接口分为三类。

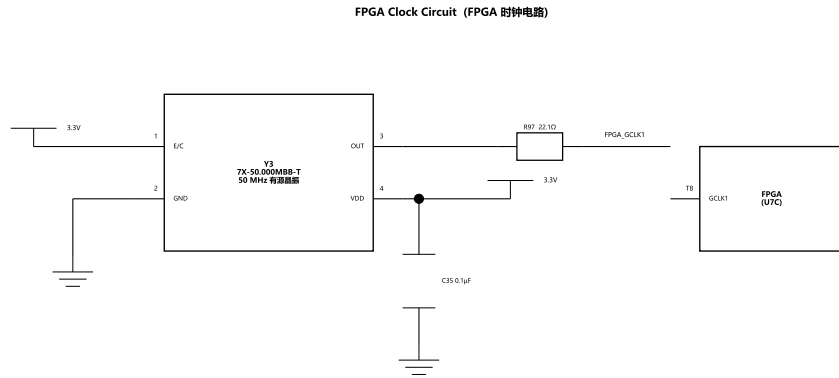


图 11 FPGA 时钟电路

UART: U2 (CH340N) 完成 USB 转串口, 其 TXD 接网络 RXD 至 FPGA F12、RXD 接网络 TXD 至 FPGA D12, 构成收发交叉。

调试/仿真器: U4 (GD32F150G8U6) 的 PA9、PA10 经 R71/R70 (0Ω) 接 FPGA 的 UART_TX (C13) 与 UART_RX (E12); PA11、PA12 经 R67/R66 (22.1Ω) 接 USB_D-、USB_D+。

DAC: FPGA 的 led[0..7] (B14/B15/B16/C15/C16/E13/E16/F16) 经 R46-R53 (20.0k) 与 R-2R 网络 (R56-R62) 送入运放 U1 (SGM321), 输出由 J1 引出。

如图 12 所示。

(三) Verilog 程序说明

本设计采用模块化 Verilog HDL 编程方式。

运动控制部分主要包括:

```
target_input → state_machine → trajectory_planner → pid_controller →
virtual_motor
```

语音部分主要包括:

```
audio_pcm_bridge → audio_vad → v2_frame_ctrl → feature_engine
feature_engine → seg_decide
audio_pcm_bridge → dir_energy
```

其中 `feature_engine` 内部由预加重、加窗、FFT、Mel、Log、DCT 等输出 13 维 MFCC; `seg_decide` 在 VAD 语音段末同时完成主人身份判决与命令判决, 替代了早期的逐帧 `speaker_verify`、`cmd_matcher` 判决路径。

最终通过:

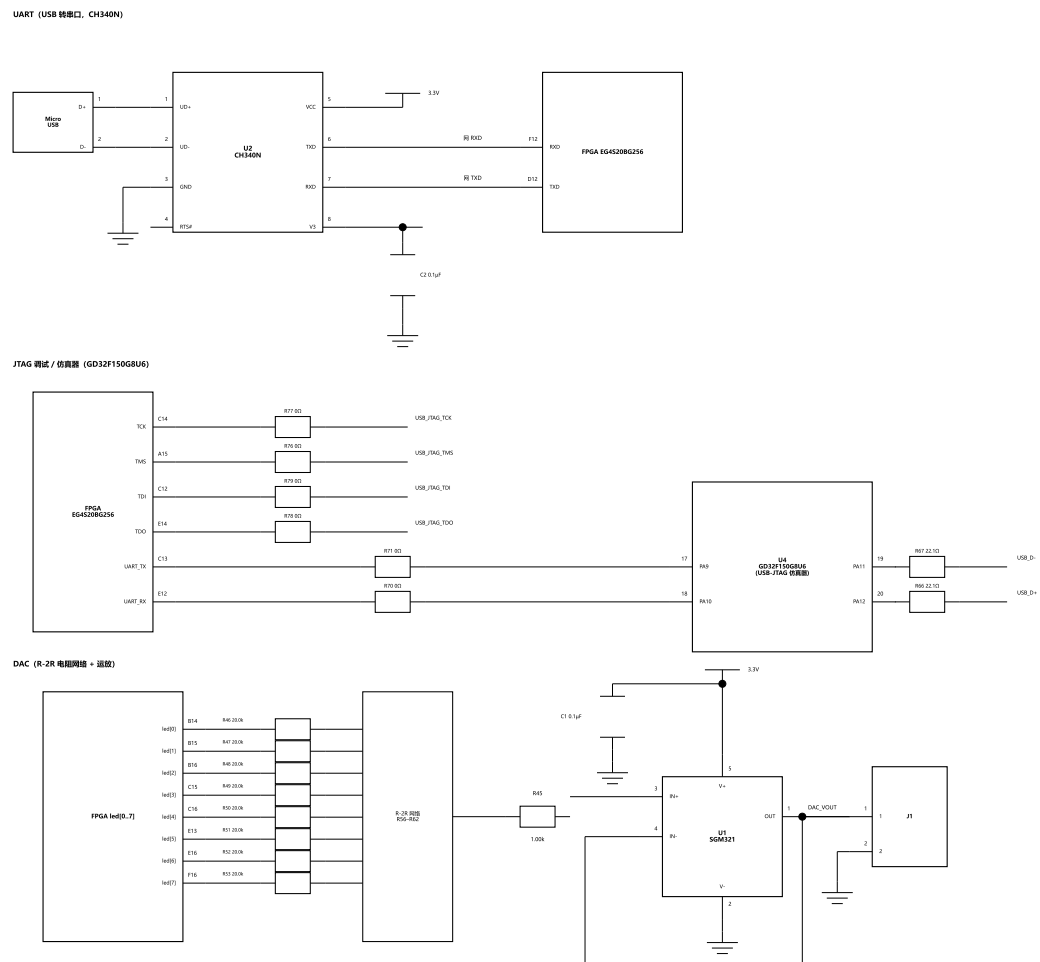


图 12 外设接口电路

decision_fsm → motion_plan → trajectory_planner → pid_controller →
virtual_motor

完成运动控制。

1. 时钟与复位设计

系统使用 50 MHz 主时钟。不同功能模块需要的低频控制节拍通过 clock-enable 产生，以降低多时钟域带来的时序分析复杂度。

2. 定点运算设计

由于 FPGA 不适合直接进行大量浮点运算，因此 MFCC 和运动控制算法均采用定点形式。

对于涉及符号变化的中间计算，本设计采用较宽的有符号中间变量，并在运算结束后进行截断和饱和处理，以避免 Verilog 中无符号数参与负数运算造成异常。

3. 系数表与模板的存储

主人声纹模板、四类命令模板以及 Hann 窗、旋转因子、Mel 基、DCT 基、Log 查找表等系数，在仿真阶段以 .mem 文件由 \$readmemh 加载。但在实际综合中发现，TD 工具对 \$readmemh/initial 方式的 ROM 初始化支持有限，若沿用该写法会导致系数表无法正确综合。为此，板上版本把这些系数改写成以 case 语句内嵌的常量形式（组合 ROM），数值与 .mem 文件逐项一致，从而在保证算法不变的前提下顺利综合上板。这是一个仿真可行不等于综合可行的典型例子。

4. UART 遥测

UART 采用 115200 baud、8N1 格式。

V1 遥测帧结构为：

[AA] [state] [target] [pos] [chk]

系统约每 100 ms 发送一帧，通过 COM8 串口被 PC 数字孪生和调试工具接收。

（四） 运行结果、存在问题分析及改进方法

1. V1 FPGA 板上运行结果

V1 系统运行流程为：

SW0 释放复位 → 输入目标角度 → KEY11 确认 → 运动 → 到位保持

首先将 sw0 上拨，使系统处于开机状态。然后通过矩阵按钮作为输入，0 至 9 为对应的数字输入，10 为 delete，11 为 enter。按矩阵按钮输入数字、输入 enter、delete 清除时，有三种音调的蜂鸣器蜂鸣作为提示音。输入完成后，数码管显示当前位置。LED0 至 2 分别显示目标设定、运动状态和到位状态。

通过 SW2 可以手动触发故障，系统进入 FAULT 状态；随后通过按键执行清除故障操作。

2. ES8388 板上运行结果

ES8388 数字接口调试过程中首先发现 I2C 配置没有生效。

原例程采用的从机地址与实际模块不一致，经板级测试后确认当前模块使用 0x11。进一步排查发现 es8388_config 中的 SLAVE_ADDR 参数覆盖了底层 I2C 驱动中的默认地址，因此需要在配置模块中进行修改。

另外，实际连线及运行后发现，无法实际接收到我的串口信号。经过一整天的反复排查发现，是所使用的 ES8388 的 DO 口和 DI 口存在实际的出入通路与丝印之间相反的问题。排查过程经历了换用 line in 口，换用杜邦线连接虚拟信号至咪头输入口，测量各点电压偏压，测量各接口的波形图等。

修改后，ES8388 配置正常，BCLK、LRCK 和数字音频数据均出现正常活动。

3. UART 丢字节问题

在 UART 联调过程中发现，早期仿真虽然 PASS，但实际通信过程中出现部分字节丢失。

进一步检查发现，uart_tx 的 busy 信号存在时序延迟，而 telemetry 模块没有等待前一个字节真正发送完成就继续发送下一字节。

修复后增加发送完成握手，最终能够正确发送完整 5 字节遥测帧。

这说明 Testbench 不能只检查模块之间的握手信号，还应该验证最终输出接口上的真实串行数据。

4. 麦克风模拟前端调试

调试初期，ES8388 的数字链路（I2C 配置、I2S 时序）已经正常，但麦克风模拟输入只有约 1 LSB 的噪声，得不到有效语音信号。当时能够确认的是：

I2C 数字配置正常；I2S 数字链路正常；但麦克风模拟前端仍有问题。

处理思路不是继续盲目修改寄存器，而是按“数字接口”“模拟前端”两部分划分问题，先确认数字链路无误，再把问题定位到模拟输入。后续按模块原理图检查

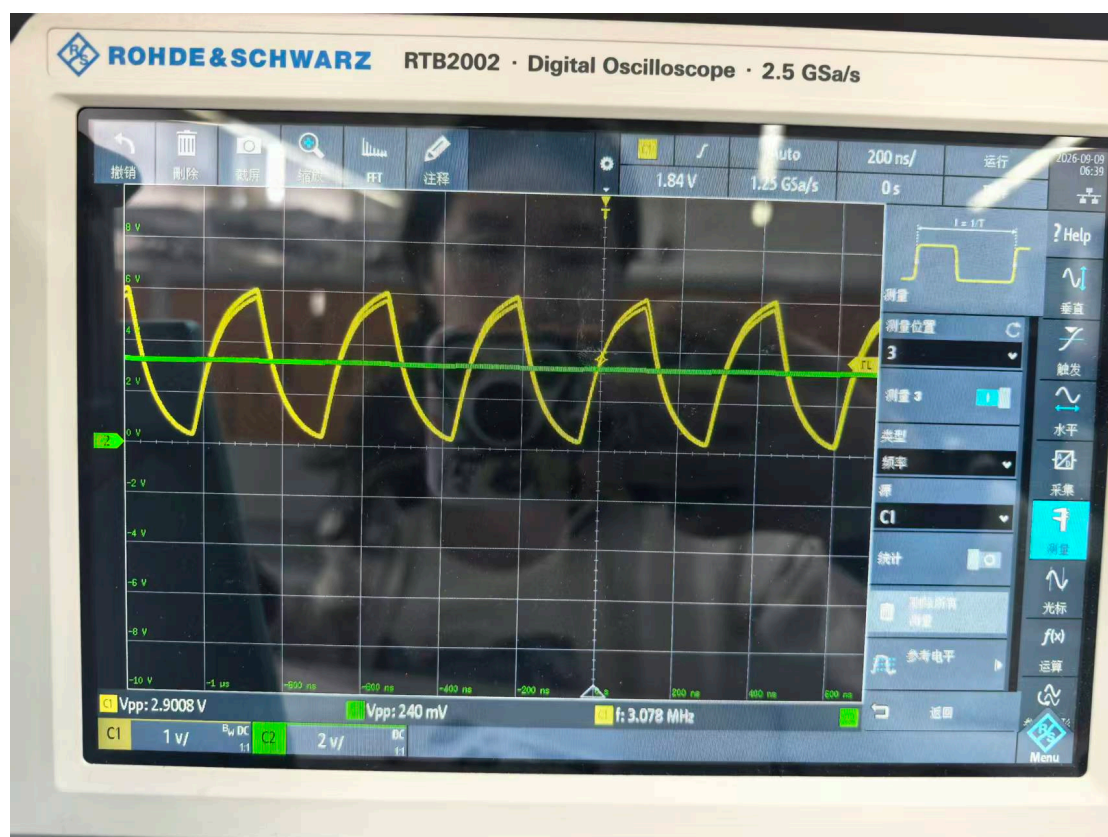


图 13 测试过程波形图

了麦克风接线、偏置与输入模式，并接入两颗咪头到左、右声道，通过“捂住单侧话筒分别说话”的方式验证左右声道相互独立。

在此基础上，用板上真实采集对 VAD 阈值做了数据定标：静音帧峰值常见约 0x03~0x0B、偶有尖峰到约 0x0D3A，说话帧峰值约 0x1C~0x65。据此取 VAD_TH_ON=0x100000、VAD_TH_OFF=0x0A0000、拖尾 32 帧，使静音不误触发、说话可靠越过。

5. 遗留问题

实测中，对麦克风说“前进”，数字孪生机械臂正确转到 90° 并经历完整状态迁移。

同时也还存在以下问题：其一，13 维 MFCC 特征对 stop/left/right 的判别能力有限，左、右存在混淆；其二，模板只在采集时的声学条件下有效，跨会话存在声学失配；其三，声纹与命令判决依赖段级均值决策，段内的逐帧信息未被充分利用。这些问题指向后续的方向——改进特征、引入归一化，或换用更鲁棒的模型。

五、 课程设计心得体会

（一） 个人分工

本课程设计主要由本人独立完成。

在项目实施过程中，本人主要负责系统总体架构设计、FPGA 模块划分、Verilog RTL 编写、ModelSim 仿真、Python Golden 参考模型、语音算法验证、ES8388 接口调试、FPGA 上板测试、UART 通信、Web 数字孪生以及最终报告撰写等工作。

整个项目按照“先建立运动控制平台，再增加语音感知”的方式逐步扩展。首先完成 V1 运动控制闭环，在确认目标输入、状态机、轨迹规划、PID 和虚拟电机均能够正常工作后，再增加 ES8388 音频接口和语音算法模块。

（二） 设计中遇到的问题及解决方法

1. 定点运算问题

在最初编写 Verilog 定点控制算法时，曾经遇到负数计算异常的问题。

例如，当无符号参数直接参与取负操作时，结果可能按照补码产生一个非常大的正数，从而导致虚拟电机位置计算异常。

针对这一问题，后续统一规定中间计算量采用有符号宽位变量，同时在各级计算之后进行限幅。

这一过程让我认识到，FPGA 设计中的“数学公式正确”并不意味着“Verilog 实现一定正确”，数据类型、位宽和符号扩展同样是电路设计的重要组成部分。

2. UART 仿真假通过问题

UART 调试是本项目中比较典型的一次工程教训。

最初 Testbench 只检查了 UART 模块收到的字节派发是否正确，没有真正检查 TX 引脚输出的串行位流，因此仿真结果显示 PASS，而实际 FPGA 与 PC 通信时却出现丢字节。

后来重新检查发送时序，发现发送模块 busy 存在两拍延迟，而上层模块没有等待发送完成。

修复握手逻辑后问题解决。

这让我认识到，Testbench 应尽可能接近真实硬件接口进行验证，而不能只验证抽象的内部信号。

3. ES8388 I2C 地址问题

ES8388 调试初期，I2C 配置无法正常生效。

经过板级排查，确认当前模块实际使用的地址为 0x11，而原例程采用 0x10。进一步检查代码后发现，真正控制地址的是 es8388_config 中的参数，而不是最底层 I2C 驱动器的默认值。

修改后数字音频链路恢复正常。

这一过程让我体会到，在硬件系统调试中，应该根据示波器、LED、UART 等实际现象逐级定位，而不能仅仅根据软件代码推测硬件行为。

4. 麦克风模拟前端问题

调试初期，ES8388 的 I2C 与 I2S 数字链路都已正常，但 ADC 数据没有出现有效语音波动。

面对这一问题，没有继续通过大量修改寄存器“碰运气”，而是先把问题分成数字接口和模拟前端两部分，确认数字链路无误后，将问题定位到麦克风模拟输入，再结合模块原理图检查输入方式、偏置和接线。

这一问题最终通过接入两颗咪头到左右声道、并验证两声道相互独立而解决。它让我体会到：硬件调试应按“先二分、再定位”的顺序逐步缩小范围，而不是在没有依据的情况下反复试参数。

5. 声纹识别数据集问题

在真实录音实验中发现，简单的 MFCC13+L1 方法在当前数据集上无法稳定区分主人和冒认者。

由于样本数量较小，而且录音内容具有较高相似性，如果直接选择一个阈值使实验结果“看起来很好”，反而会掩盖真实问题。

因此后续引入 CAM++ 进行离线参考，同时使用 leave-one-out 方法进行评估。

这让我认识到，工程项目中的实验数据必须真实反映系统能力，不能为了得到漂亮的数字而选择不合理的测试方式。

6. 逐帧决策与段平均决策

这是本项目最有价值的一次教训。板级实测时，声纹认证始终无法通过，命令也恒为“停止”。排查后发现两个层面的根因：一是命令匹配器对每一帧都会输出结果（包括静音帧），静音帧总是选中幅度最小的“停止”模板；二是主人和陌生人的逐帧距离分布几乎完全重叠，单一阈值无法分开。

把决策粒度从“逐帧”改为“分段平均”（对整段语音的 MFCC 求均值再判决）后，两个问题同时解决：段均值抑制了随机波动，主人与陌生人的距离清晰分离；命令判决也只在完整语音段上进行。

这让我认识到，在资源受限的轻量级系统中，合理的算法结构和决策粒度，有时比单纯追求特征精度更重要。

7. 跨会话声学失配

在后续测试中又发现，即使模板和阈值都正确，只要测试时的录音条件（距离、音量、位置）与建立模板时不同，识别就会失效——同一词语的 MFCC 前几维甚至出现符号翻转。

这说明基于模板的方法只对“采集时的声学条件”有效。工程上的应对是先保证同条件采集与测试，长期则需引入能量归一化或更鲁棒的特征。这个现象也解释了为什么数据驱动、条件一致的采集对这类系统至关重要。

（三） 课程设计收获

通过本次课程设计，我对 FPGA 系统设计的认识从单个 Verilog 模块进一步扩展到了完整系统。

在硬件方面，我学习了 FPGA 时钟、复位、GPIO、UART、I2C、I2S 和 BRAM 等接口的设计与调试方法。

在算法方面, 我将 MFCC 等原本在软件中运行的数字信号处理算法逐步转换成 FPGA 定点逻辑, 并通过 Python Golden 与 RTL 进行逐位验证。

在工程方法方面, 我形成了设计规则、模块化、单元测试、Golden 验证、ModelSim、FPGA 上板的开发流程。

特别是在 UART、ES8388 和语音决策调试过程中, 我认识到仿真正确并不一定意味着真实硬件一定正确, 必须将软件模型、RTL 仿真和实际硬件测试结合起来。

最终, 本项目实现了从真实语音输入到机器人运动、再到 PC 数字孪生显示的完整板级闭环, 这是整个课程设计中最有成就感的一步。当然, 系统在特征判别力、跨会话鲁棒性等方面仍有明显局限, 但正是这些在实践中暴露的真实问题, 让我更加理解电子系统从理论设计走向真实硬件时所面临的困难, 也构成了本次课程设计最重要的收获。

六、 参考资料及网站

- [1] 安路科技 (Anlogic). EG4S20 (EAGLE) 系列 FPGA 数据手册与使用指南 [Z/OL].
- [2] 大拇指 (DMZ_Anlogic). EG4S20 开发板配套原理图与官方例程 [Z].
- [3] Everest Semiconductor. ES8388 V1.1 Low Power Stereo Audio CODEC Datasheet[Z].
- [4] Davis S B, Mermelstein P. Experiments in Automatic Speech Recognition of Digits[J]. IEEE Transactions on Acoustics, Speech, and Signal Processing, 1980.
- [5] 阿里达摩院 / ModelScope. speech_campplus_sv_zh-cn_16k-common: CAM++ 说话人验证模型 [Z/OL].
- [6] MDN. Web Serial API 开发文档 [Z/OL].
- [7] Anlogic TangDynasty 综合布局布线工具与 Mentor ModelSim 仿真用户手册 [Z/OL].

七、 设计过程中 AI 应用实例

在本课程设计过程中, 人工智能主要作为辅助分析、资料整理和问题定位工具使用。项目中的总体方案设计、硬件架构搭建、RTL 代码编写、仿真验证以及 FPGA 实物调试均由本人独立完成。AI 的作用主要体现在提高开发效率、辅助分析复杂问题以及提供设计优化思路, 而最终设计方案和实验结果均经过人工判断和实际验证。

（一） Verilog 程序设计辅助

在 FPGA 系统开发过程中，AI 主要用于辅助理解硬件设计方法以及提供部分模块的代码参考。针对一些具有明确结构和较强重复性的基础模块，例如：时钟使能模块；LED 与数码管显示控制；矩阵键盘扫描模块；UART 通信模块；FFT、Mel 滤波、DCT-II 等信号处理模块；声纹识别与命令匹配相关接口模块。

AI 可用于提供初始设计思路或代码框架，但实际工程实现过程中，需要根据 FPGA 资源限制、接口时序要求以及系统整体架构进行人工修改。

例如，在 UART 通信、MFCC 特征提取以及 ES8388 音频接口设计过程中，本人根据实际硬件连接关系重新设计模块接口，并通过 ModelSim 仿真、Python Golden 模型对比以及 FPGA 上板测试逐步验证。

因此，AI 输出仅作为设计参考，最终 RTL 代码、模块连接关系以及系统结构均由本人完成设计与调整。

（二） 算法分析与验证辅助

在语音信号处理部分，AI 主要用于辅助分析算法流程以及检查 Python 验证程序中的数据处理逻辑。

例如，在 MFCC 特征提取过程中，AI 可辅助解释预加重、加窗、FFT、Mel 滤波、Log 运算以及 DCT-II 等算法步骤，帮助快速定位算法实现中的理论问题。

但实际工程中，算法参数选择、定点化处理方式以及 FPGA 实现方案均需要结合硬件资源和实时性要求进行人工设计。

最终结果通过 Python Golden 模型与 FPGA RTL 输出进行比对，确保硬件实现与理论算法一致。

（三） 硬件调试过程中的 AI 辅助

在 FPGA 实际调试阶段，AI 主要用于辅助分析实验现象。

例如，在 UART 通信调试过程中，通过分析串口波形、发送时序以及 RTL 状态机运行情况，最终定位到 busy 信号握手和发送流程问题，我再通过修改 RTL 逻辑完成修复。

在 ES8388 音频模块调试过程中，AI 用于辅助分析 I2S 波形、I2C 配置流程以及可能的硬件连接问题。但最终问题定位仍依靠本人对原理图、芯片手册以及示波器测量结果进行综合判断。

通过实际排查，发现初始问题与 I2S 数据方向连接错误以及寄存器配置有关，并通过修改接口连接和配置逻辑使音频数据能够正常采集。

AI 不直接替代工程设计过程，而是作为提高调试效率的辅助工具。

（四） AI 应用的限制

在 FPGA 硬件设计过程中，AI 生成的信息仍存在一定局限性。

例如，如果缺少官方原理图、芯片手册或 FPGA 引脚约束文件，AI 可能会根据已有经验进行推测，而这些推测在实际硬件环境中可能并不正确。

因此，本项目在使用 AI 辅助开发时遵循以下原则：

1. FPGA 引脚约束、外设连接关系以及通信协议参数必须以官方资料 and 实际硬件为依据，不采用未经验证的推测结果；
2. 所有 RTL 代码修改必须经过仿真验证，并结合 FPGA 实际运行结果确认；
3. 所有实验结论必须来源于实际测试数据，而非 AI 分析结果。

综上，AI 在本项目中主要承担辅助设计、辅助分析和辅助调试的角色，而系统架构设计、硬件实现、程序调试以及最终实验验证均由本人独立完成。

图 目 录

1	系统总体结构框图	4
2	MFCC 特征提取流水线	16
3	声纹认证与段级决策波形（基于板上真实采集数据）	17
4	KWS 命令识别波形（基于板上真实采集数据）	18
5	Python Golden 与 RTL 逐位比对（真实录音，16 帧 ×13 维全部 diff=0）	19
6	声纹评估 ROC 与 DET 曲线（真实录音：MFCC13+L1 与 CAM++， full 与 leave-one-out）	20
7	基于板上真实采集数据的 Who + What 端到端联合决策波形	21
8	Web 数字孪生系统界面	22
9	FPGA 供电电路	25
10	FPGA AS 配置电路	26
11	FPGA 时钟电路	27
12	外设接口电路	28
13	测试过程波形图	31